CODEBEARERS · STUDY PACK

CodeChum National Programming Challenge 2026

Python reviewer: rules, must-know functions, data structures, techniques, and step-by-step breakdowns of every practice and past problem.

26
problems broken down step by step (10 practice + 16 from last year)

15
solving techniques with templates

14
new drill problems with answer key

How to use this in the days before the contest

When	Do this
Day 1	Part 1 (rules) + Part 2 (Python). Type every snippet once from memory.
Day 2	Part 3 + Part 4. For each technique, re-solve its "Seen in" problem **without looking**.
Day 3–4	Part 5: cover the solution, read steps 1–5, write code yourself, then compare.
Day 5	Part 6 drills under a timer (15 min each). Check with the answer key.
Night before	Only Part 1 + the final checklist on the last page. Sleep.

Every solution in this PDF was run against every sample shown, cross-checked against brute force on random inputs, and timed at the maximum input size. Source files: `Reviewer/solutions/`.

Contents

1 · Contest-Day Rules 3

2 · Python You Must Have Memorised 5

3 · Data Structures 8

4 · Techniques Playbook 10

5 · Problem Breakdowns 13

5A · Practice problems (given to us)

P01 · Theater Seat Total Easy 14
P02 · Fix One Street Lamp Medium 16
P03 · Rotting Fruit Crate Medium–Hard 18
P04 · Calculator Without Zeros Easy 21
P05 · Longest Word Length Easy 22
P06 · Round to the Nearest Ten Easy 23
P07 · Warmer Than Yesterday Easy 25
P08 · Library Late Fines Easy 27
P09 · Plus-Minus Range Sum Easy 29
P10 · Tiered Water Bill Easy 30

5B · Last year — Preliminaries

L01 · Alternating Character Pattern Easy–Medium 31
L02 · Peak-Valley Difference Easy–Medium 33
L03 · Palindromic Prime Counter Medium 35
L04 · Digit Product Sum Series Medium 37
L05 · Cyclic Array Shift Easy–Medium 39
L06 · Pascal's Triangle Easy–Medium 41
L07 · Stable Sequence Reduction Medium 43
L08 · Maximal Square of Ones Hard 45

5C · Last year — Semifinals

S01 · Alternating Factorial Sum Easy 47
S02 · Digital Root Easy 48
S03 · Positional Cipher Medium 49
S04 · Digit Sum Power Sum Easy–Medium 51
S05 · Roman to Integer Easy–Medium 52
S06 · Smallest Missing Subarray Sum Hard 53
S07 · ZigZag Conversion Medium 55
S08 · Largest Symmetric Subgrid Hard 57

6 · Drill Set (14 problems) 59

7 · Drill Answer Key 66

8 · Final Checklist 71

1 Contest-Day Rules

Read this first. These rules matter more than any algorithm.

The #1 reason correct code loses points on CodeChum

CodeChum checks your output **character by character**. The **prompt text** is part of the output. A missing space after a colon, a lowercase letter, or an extra line = **wrong answer**, even if your algorithm is perfect.

Rule 1 — Copy the prompts and labels exactly

The problem says…	You write…
read after the prompt `Enter number:`	`n = int(input("Enter number: "))` (note the space after the colon)
output `Display: D`	`print(f"Display: {d}")`
program prints `Enter the crate:` **on its own line**, then reads R lines	`print("Enter the crate:")` then `input()` with **no prompt** for each line
two values on one line after `Enter rows and columns:`	`R, C = map(int, input("Enter rows and columns: ").split())`

How to read a CodeChum sample box

`Enter number: 40507` — the part `40507` is what the **user typed**. Your program prints only `Enter number: `. Python's `input("Enter number: ")` does exactly that. In this reviewer the typed part is highlighted in every sample run.

Rule 2 — The constraint bullets ARE the hidden test cases

CodeChum shows 2–3 samples but grades on hidden tests. The extra bullets in **Constraints** tell you exactly what those hidden tests are. From the practice set:

Constraint bullet	Hidden test you must pass
"R may be 1, in which case only the first row exists."	R = 1
"There may be no rotten fruit; the answer is then 0."	grid with no `R`
"Every lamp may be broken."	`0000` → 1
"Rounding up may add a digit (when N ends in several 9s)."	999 → 1000
"N may have no zeros at all."	98765 → 98765
"The count may be 0 (such as when N = 1)."	N = 1

Habit: before you submit, turn every bullet into a test case and run it with **Run code**.

Rule 3 — The 5-step loop for every problem

Step	Do this	Time
1 READ	Read the statement twice. Underline every prompt string and output label.	2 min
2 CLASSIFY	Which pattern is it? (see Part 4). Check N in the constraints → decides the speed you need.	1 min
3 EDGE LIST	Write the constraint bullets as test inputs in a comment at the top of your file.	1 min
4 CODE	Start from the I/O template. Read input exactly as described, then solve.	5–15 min
5 TEST	Run all samples, then every edge case, then one maximum-size input. Then submit.	3 min

Rule 4 — Choose the algorithm from N

Python does roughly **10 million simple operations per second**. Look at the biggest number in the constraints and pick an approach that fits:

Biggest N	OK approach	Example from our problem sets
N ≤ 10	anything, even trying all orders (permutations)	—
N ≤ 100	$O(N^3)$ — triple loops are fine	Street Lamp (try every lamp), Stable Sequence
N ≤ 2,000	$O(N^2)$ — double loop	Smallest Missing Subarray Sum
N ≤ 500 × 500 grid	O(N·M) — visit each cell a few times	Maximal Square (DP)
N ≤ 10^5 – 10^6	O(N) or O(N log N) — one loop, sorting, sieve	Palindromic Primes, Plus-Minus Sum
N ≤ 10^9 or more	O(1) formula or O(digits) — never loop up to N	Round to Ten, Calculator, Digital Root

Rule 5 — Points strategy

- In the practice set **every item was worth 10 points**. Equal points → solve the **easiest first**, not in order.
- First 5 minutes: skim **all** problems and label each **E / M / H**. Do every E, then M, then H.
- Stuck for more than ~15 minutes? Submit what you have (if scoring is per test case, a brute force still earns points) and move on. Come back later.
- **Check the editor language before typing.** The practice screens showed `main.c` on most items — make sure the file says `script.py` / Python is selected.
- Never leave an item blank. Even reading the input and printing the label right can pass an edge test (e.g. answer 0).

Rule 6 — Team split (adjust to your team size)

Member	First pass	Then
A — fastest typer	All **Easy** items, top to bottom (formula, loops, strings)	Helps test others' edge cases
B — strongest on strings/lists	**Medium** string / array items (ciphers, patterns, shifts)	Takes the next Medium
C — strongest on grids/logic	**Hard** items (grids, DP, BFS, simulations)	Partial-credit brute force if stuck

If the contest is individual, use the same order yourself: Easy → Medium → Hard.

The I/O template — memorise this shape

template.py

```python
# 1) READ  - copy prompt text exactly (including the space after ':')
n = int(input("Enter N: "))
arr = list(map(int, input("Enter the numbers: ").split()))

# 2) SOLVE - plain variables, no fancy tricks needed
answer = 0
for x in arr:
    answer += x

# 3) PRINT - copy the output label exactly
print(f"Result: {answer}")
```

2 Python You Must Have Memorised

No internet, no autocomplete. These must come out of your fingers.

2.1 Reading input — every shape you will meet

Input looks like	Code
one integer	`n = int(input("Enter N: "))`
two integers on one line `3 3`	`a, b = map(int, input("...: ").split())`
N integers on one line	`arr = list(map(int, input("...: ").split()))`
an int and a float on one line `4 2.0`	`p = input().split(); n = int(p[0]); t = float(p[1])`
a sentence (keep spaces)	`s = input("Enter sentence: ")` — do **not** split
a 'number' you treat as text (`0011`)	`s = input().strip()` — `int()` would delete leading zeros
R lines of characters (grid)	`grid = [list(input().strip()) for _ in range(R)]`
R lines of numbers (matrix)	`mat = [list(map(int, input().split())) for _ in range(R)]`
a single character `L` / `R`	`d = input().strip().upper()`
numbers that may wrap onto several lines	`a = []` then `while len(a) < n: a += map(int, input().split())`

2.2 Printing output

Goal	Code	Prints
label + value	`print(f"Total: {t}")`	`Total: 52`
label + value (comma form)	`print("Total:", t)`	`Total: 52` (adds one space)
list space-separated	`print(*lst)`	`1 2 3`
label + list	`print("Shifted array:", *lst)`	`Shifted array: 3 4 5 1 2`
join strings	`" ".join(map(str, lst))`	`1 2 3`
2 decimals	`f"{x:.2f}"`	`10.50`
no newline at end	`print(x, end="")`	
no space between items	`print(a, b, sep="")`	`ab`
pad to width 5	`f"{x:>5}"` / `f"{x:<5}"`	`42`
each row of a grid	`for row in grid: print("".join(row))`	`RRF`

2.3 Strings

Code	What it does
`s.split()`	words, splits on any whitespace (safer than `split(' ')`)
`" ".join(words)`	glue a list of strings back together
`s.replace('0', '')`	remove / replace every occurrence
`s.strip()`	remove spaces and newline at both ends
`s.lower()` , `s.upper()`	change case (for case-insensitive checks)
`ch.isalpha()` , `ch.isdigit()` , `ch.isupper()` , `ch.islower()` , `ch.isspace()`	classify one character

Code	What it does
`s[::-1]`	reverse — palindrome test: `s == s[::-1]`
`s[i]`, `s[a:b]`, `s[-1]`	index, slice, last character
`s.count('a')`, `s.find('ab')`, `'ab' in s`	count, first position (-1 if none), contains
`ord('a')` → 97, `chr(97)` → `'a'`	letters ↔ numbers (ciphers)
`(ord(ch) - ord('a') + k) % 26`	position 0–25 after shifting by k, wraps around
`str(n)`, `len(str(n))`, `int(d) for d in str(n)`	digits of a number

2.4 Lists

Code	What it does
`a.append(x)`, `a.pop()`, `a.pop(0)`	add to end, remove from end (fast), remove from front (slow → use deque)
`a[k:] + a[:k]`	rotate left by k
`a[::-1]`, `a.reverse()`, `reversed(a)`	reverse (copy / in place / iterator)
`sorted(a)`, `sorted(a, reverse=True)`, `a.sort()`	sort (copy / descending / in place)
`sorted(a, key=len)`, `key=lambda p: (-p[1], p[0])`	sort by a rule; tuple key = tie-breakers
`sum(a)`, `min(a)`, `max(a)`, `max(a, key=len)`	totals and extremes
`a.index(x)`, `a.count(x)`, `x in a`	position, count, membership (O(N) — use a set for speed)
`for i, x in enumerate(a)`	index and value together
`for x, y in zip(a, a[1:])`	every pair of neighbours
`[x * 2 for x in a if x > 0]`	list comprehension: map + filter
`[[0] * m for _ in range(n)]`	**correct** 2D grid
`list(zip(*m))`	transpose a matrix; `zip(*m[::-1])` rotates 90° clockwise

2.5 Math

Code	Notes
`a // b`, `a % b`, `divmod(a, b)`	integer division, remainder, both at once
`n % 10`, `n // 10`	last digit, remove last digit
`abs(x)`, `x ** y`, `pow(a, b, m)`	absolute, power, fast power mod m
`math.isqrt(n)`	exact integer square root (prime checks)
`math.gcd(a, b)`, `math.lcm(a, b)`	gcd / lcm (lcm needs Python 3.9+)
`math.factorial(n)`, `math.comb(n, k)`, `math.perm(n, k)`	n!, nCk, nPk
`float('inf')`, `-float('inf')`	start values for min / max searches
`(n + 5) // 10 * 10`	round to nearest 10 (half rounds up) — safe integer way
`-(-a // b)`	ceiling division without floats
Python ints never overflow	20! or 10^{18} is fine — no `long long` worries

2.6 Loops you should be able to write without thinking

loop patterns

```
for i in range(a, b + 1):          # a..b INCLUSIVE
for i in range(n - 1, -1, -1):     # n-1 down to 0
for i in range(1, n):              # compare a[i] with a[i-1]
for i in range(1, n - 1):          # both neighbours exist (peaks/valleys)
for r in range(R):                 # every cell of a grid
    for c in range(C):
        ...
while n > 0:                       # every digit of n, right to left
    d = n % 10
    n //= 10
while True:                        # simulate until nothing changes
    changed = False
    ...
    if not changed:
        break
```

2.7 Python traps that silently give wrong answers

Trap	What happens	Fix
`round(2.5)` / `round(25, -1)`	gives `2` / `20` — Python rounds .5 to the **even** number	integer math: `(n + 5) // 10 * 10`
`grid = [[0]*m]*n`	every row is the **same** list; changing one changes all	`[[0]*m for _ in range(n)]`
`a[i-1]` when `i = 0`	`a[-1]` is the **last** element — no error, wrong answer	start loops at 1 / check `i > 0`
`int("0011")`	becomes `11` — leading zeros gone	keep it as a string
`"a b".split(" ")`	`['a', '', 'b']` — empty words	use `split()` with no argument
`-7 // 2`	`-4` (floors toward −∞)	be careful with negatives
`-1 % 26`	`25` — Python's `%` is never negative (good for ciphers!)	use it on purpose
`int("2.0")`	crashes (`ValueError`)	`float("2.0")` first
naming a variable `sum`, `max`, `list`, `str`	you lose the built-in function	use `total`, `best`, `arr`, `s`
deep recursion	crashes after ~1000 calls	write it with a loop / queue instead
`0.1 + 0.2 == 0.3`	`False` (float error)	compare with `abs(a-b) < 1e-9`, or use integers

3 Data Structures

What you must know vs. what is nice to have.

3.1 MUST know (used in our problem sets)

Structure	Use it when…	Key operations	Seen in
list	ordered values, arrays, results	`append`, index, slice, `sort`, `sum`	almost everything
2D list (grid)	rows × columns, maps, matrices	`g[r][c]`, bounds check `0 <= r < R`	Rotting Fruit, Maximal Square, Symmetric Subgrid
str	text, digits of a number	`split`, `replace`, slicing, `[::-1]`	Calculator, Longest Word, Cipher, ZigZag
dict	map a key to a value (lookup table)	`d[k]`, `d.get(k, 0)`, `for k, v in d.items()`	Roman to Integer
set	fast 'have I seen this?', remove duplicates	`add`, `in` (O(1)), `len(set(a))`	Smallest Missing Sum, distinct digits
deque	queue for BFS (fast pop from the front)	`append`, `popleft`	Rotting Fruit (BFS version)

collections — the three you need

```
from collections import deque, Counter, defaultdict

q = deque([(0, 0)]); q.append((1, 2)); r, c = q.popleft()     # queue
cnt = Counter("banana")                # Counter({'a': 3, 'n': 2, 'b': 1})
cnt.most_common(1)                     # [('a', 3)]
groups = defaultdict(list)             # missing key -> empty list automatically
groups["even"].append(4)
```

3.2 GOOD to have (common in harder rounds)

Structure / idea	Use it when…	Code
stack (list)	brackets, undo, 'most recent first'	`st.append(x)`, `st.pop()`, `st[-1]`
prefix sums	many 'sum from L to R' questions	`pre[i+1] = pre[i] + a[i]`; sum = `pre[R+1] - pre[L]`
sliding window	best sum / count over K consecutive items	add new item, subtract the one leaving
heapq	repeatedly take the smallest item	`heapq.heappush(h, x)`, `heapq.heappop(h)`
bisect	position in a sorted list	`bisect.bisect_left(a, x)`
tuple as dict key	grid cells, pairs	`seen.add((r, c))`
itertools	all pairs / orders / combos	`combinations(a, 2)`, `permutations(a)`, `product(a, b)`

3.3 Speed facts that decide pass / fail

Operation	Cost	Lesson
`x in list`	O(N) — scans everything	inside a loop → $O(N^2)$. Use a **set**.
`x in set`, `d[k]`	O(1)	default choice for lookups
`list.pop(0)`, `list.insert(0, x)`	O(N)	use `deque.popleft()` for queues
`s += ch` on strings in a loop	fine for ≤ 10^4 chars	for huge strings, collect in a list and `''.join`
`sorted(a)`	O(N log N)	10^6 items ≈ 1 second

You can skip this week

Linked lists, binary trees, Dijkstra, segment trees, tries. Nothing in last year's 16 problems or this year's 10 practice problems needed them. Spend the time on Parts 4 and 5.

4 Techniques Playbook

15 patterns that cover all 26 problems.

Each pattern: **how to spot it** in the wording, a **template**, and **where it appeared**. When you read a new problem, scan this list and ask "which one is this?"

T1 Direct formula / simple loop

Spot it: "Each row has 2 more…", "first 10 cost…, beyond that…", "add the first, subtract the second…"

```
total = 0
for i in range(1, n + 1):
    total += term(i)          # or find the closed formula
```

Seen in: Theater Seats, Water Bill, Plus-Minus Sum, Alternating Factorial

T2 Digit manipulation

Spot it: "digits of N", "remove zeros", "sum of digits", "palindromic number", N up to 10^9

```
s = str(n)                        # easiest: treat as text
digit_sum = sum(int(d) for d in s)
count = len(s)
is_pal = s == s[::-1]
# math way:  while n > 0: d = n % 10; n //= 10
```

Seen in: Calculator Without Zeros, Digital Root, Digit Sum Power, Digit Product Series

T3 String processing & classification

Spot it: "words separated by spaces", "vowels/consonants", "ignore non-letters", "case-insensitive"

```
words = s.split()
letters = [ch.lower() for ch in s if ch.isalpha()]
is_vowel = ch in "aeiou"
```

Seen in: Longest Word, Alternating Character Pattern, Roman to Integer

T4 Character shifting (ciphers)

Spot it: "shift each letter by…", "wraps around", "Caesar"

```
base = ord('a') if ch.islower() else ord('A')
new = chr((ord(ch) - base - shift) % 26 + base)
```

Seen in: Positional Cipher

T5 Neighbour comparison

Spot it: "higher than the day before", "greater than both neighbours"

```
for i in range(1, n):             # a[i] vs a[i-1]
    if a[i] > a[i - 1]: ...
for i in range(1, n - 1):         # a[i] vs both sides
    if a[i] > a[i-1] and a[i] > a[i+1]: ...
```

Seen in: Warmer Than Yesterday, Peak-Valley Difference

T6 Runs / streaks & “try every option”

Spot it: “longest stretch”, “consecutive”, “choose exactly one to change, best result”

```
best = cur = 0
for x in a:
    cur = cur + 1 if ok(x) else 0
    best = max(best, cur)
# try every option: for each candidate, compute result, keep max
```

Seen in: Fix One Street Lamp

T7 Clamping & tiers

Spot it: “never more than 50”, “no fine for 7 days or less”, “first 10 at …, the rest at …”

```
fine = min(50, max(0, d - 7) * 5)
bill = min(u, 10) * 15 + max(u - 10, 0) * 25
```

Seen in: Library Late Fines, Tiered Water Bill

T8 Grid neighbours with bounds

Spot it: grid / matrix + “above, below, left, right”

```
DIRS = ((-1, 0), (1, 0), (0, -1), (0, 1))
for dr, dc in DIRS:
    nr, nc = r + dr, c + dc
    if 0 <= nr < R and 0 <= nc < C:      # ALWAYS check before g[nr][nc]
        ...
```

Seen in: Rotting Fruit Crate, Largest Symmetric Subgrid

T9 Multi-source BFS (spreading)

Spot it: “every minute it spreads to neighbours”, “minimum steps”, “at the same time from several places”

```
from collections import deque
q = deque(all_start_cells); dist = {cell: 0 for cell in all_start_cells}
while q:
    r, c = q.popleft()
    for each valid neighbour (nr, nc) not in dist:
        dist[(nr, nc)] = dist[(r, c)] + 1
        q.append((nr, nc))
```

Seen in: Rotting Fruit Crate (fast version), drill D04 Island Counter

T10 Simulate until stable

Spot it: “repeat until no more changes”, “all changes happen at the same time”

```
while True:
    changes = [...]          # 1) FIND all changes without applying them
    if not changes: break
    apply(changes)           # 2) APPLY all at once
    steps += 1
```

Seen in: Rotting Fruit Crate, Stable Sequence Reduction

T11 Primes: sieve

Spot it: "count primes in a range", R up to 10^5–10^6

```
is_p = [True] * (R + 1); is_p[0] = is_p[1] = False
for i in range(2, int(R ** 0.5) + 1):
    if is_p[i]:
        for j in range(i * i, R + 1, i):
            is_p[j] = False
```

Seen in: Palindromic Prime Counter

T12 Subarray sums / prefix sums / sliding window

Spot it: "contiguous subarray", "sum from L to R", "K consecutive days"

```
for i in range(n):              # every subarray starting at i
    s = 0
    for j in range(i, n):
        s += a[j]                 # s = sum(a[i..j]) in O(1) per step
```

Seen in: Smallest Missing Subarray Sum, drills D05–D06

T13 Dynamic programming on a grid / table

Spot it: "largest square", "number of paths", each answer built from smaller neighbours

```
dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])   # max square
row = [1] + [row[k] + row[k+1] for k in range(len(row)-1)] + [1]   # Pascal
```

Seen in: Maximal Square of Ones, Pascal's Triangle, drill D11

T14 Rotation, wrap-around & bouncing indexes

Spot it: "cyclic", "wrap around", "K can be larger than N", "zigzag"

```
k %= n
left = a[k:] + a[:k]
right = a[n - k:] + a[:n - k]
# bouncing row: if r == 0: step = 1; elif r == rows-1: step = -1; r += step
```

Seen in: Cyclic Array Shift, ZigZag Conversion

T15 Brute force + pruning

Spot it: "largest ... such that ...", small enough N, no obvious formula

```
for size in range(N, 0, -1):          # biggest first
    for every position:
        if cheap_check() and full_check():
            print(size); exit
```

Seen in: Largest Symmetric Subgrid

5 Problem Breakdowns

26 problems × the same 9 thinking steps.

Every breakdown follows the **same 9 steps**. Train yourself to run these steps in your head on Saturday — that's the real skill, not the individual answers.

1. **What it really asks**
2. **Exact I/O (copy these strings)**
3. **Spot the pattern**
4. **Hidden test cases (from the constraints)**
5. **Approach — before you code**
6. **Hand trace**
7. **Python solution**
8. **Will Python be fast enough?**
9. **Common mistakes & likely twists**

How to study a breakdown

Cover steps 6–9 with a sheet of paper. Read steps 1–5, write the code yourself in an editor, run the samples, **then** uncover and compare. If yours differs, find out why.

Difficulty map

Level	Practice set	Last year
Easy	Theater, Calculator, Longest Word, Round, Warmer, Library, Plus-Minus, Water Bill	Alternating Factorial, Digital Root, Digit Sum Power
Medium	Street Lamp, Rotting Fruit	Alternating Pattern, Peak-Valley, Palindromic Prime, Digit Product, Cyclic Shift, Pascal, Stable Sequence, Cipher, Roman, ZigZag
Hard	—	Maximal Square, Smallest Missing Subarray Sum, Largest Symmetric Subgrid

What this tells us about Saturday

The practice set is mostly Easy, but last year's rounds had real Medium/Hard problems (DP, sieve, grid search). **Bank every Easy item fast and correctly**, then spend the remaining time on one or two Hard ones.

PRACTICE · ITEM 1 · P01

Theater Seat Total

Easy | T1 Formula / loop

1 What it really asks

Add up R numbers: S, S+2, S+4, … (each row has 2 more seats than the one before).

2 Exact I/O (copy these strings)

INPUT `Enter seats in first row:` → S (read FIRST)

INPUT `Enter number of rows:` → R

OUTPUT `Total seats: T`

Sample runs highlighted = what the user types

Sample 1
```
Enter seats in first row: 10
Enter number of rows: 4
Total seats: 52
```

Sample 2
```
Enter seats in first row: 1
Enter number of rows: 1
Total seats: 1
```

Sample 3
```
Enter seats in first row: 5
Enter number of rows: 2
Total seats: 12
```

3 Spot the pattern

"every row after it has **2 more** than the row in front" = an **arithmetic sequence**. Either loop R times, or use the formula.

4 Hidden test cases (from the constraints)

- R = 1 → answer is just S
- S = 1, R = 1 → 1
- max: S = 1000, R = 1000 → 1,999,000 (no overflow in Python)

5 Approach — before you code

1. Read S, then R (same order as the prompts).
2. Start `seats = S`, `total = 0`.
3. Repeat R times: `total += seats`, then `seats += 2`.
4. Print `Total seats: {total}`.

6 Hand trace

```
S = 10, R = 4
row 1: 10   total = 10
row 2: 12   total = 22
row 3: 14   total = 36
row 4: 16   total = 52
formula check: S*R + R*(R-1) = 40 + 12 = 52  ✓
```

7 Python solution

solutions/P01_theater_seat_total.py

```python
S = int(input("Enter seats in first row: "))
R = int(input("Enter number of rows: "))

# Rows have S, S+2, S+4, ... seats -> loop and add
total = 0
seats = S
for _ in range(R):
    total += seats
    seats += 2

# One-line formula version: total = S*R + R*(R-1)
print(f"Total seats: {total}")
```

8 Will Python be fast enough?

R ≤ 1000 → 1000 steps. Instant.

9 Common mistakes & likely twists

- ✗ Reading R before S — the prompt order is S first.
- ✗ `range(R + 1)` adds one extra row.
- ✗ Printing `Total seats:52` (missing space).

↪ "each row has K more" → `S*R + K*R*(R-1)//2`.

↪ "each row has double the seats" → geometric: loop with `seats *= 2`.

PRACTICE · ITEM 2 · P02

Fix One Street Lamp

Medium | T6 Runs / try every option

1 What it really asks

Turn exactly one `0` into `1` so that the longest block of consecutive `1`s is as long as possible.

2 Exact I/O (copy these strings)

INPUT `Enter lamps:` → one word like `1011100111101`

OUTPUT `Longest lit stretch: L`

Sample runs highlighted = what the user types

Sample 1
```
Enter lamps: 1011100111101
Longest lit stretch: 6
```

Sample 2
```
Enter lamps: 111011101110111
Longest lit stretch: 7
```

Sample 3
```
Enter lamps: 0000
Longest lit stretch: 1
```

3 Spot the pattern

"repair **exactly one** … with the **best choice**" = **try every option, keep the best**. Only 100 lamps, so brute force is safe. The statement's tip even gives the method: left run + 1 + right run.

4 Hidden test cases (from the constraints)

- every lamp broken `0000` → 1
- broken lamp at either end `0111` → 4, `1110` → 4
- a single lamp `0` → 1
- several repairs tie (answer is still just the length)

5 Approach — before you code

1. Keep the input as a **string** (never `int()` — `0011` would become `11`).
2. For every position `i` where the lamp is `0`:
3. count consecutive `1`s going left from `i-1`, and going right from `i+1`.
4. candidate = left + 1 + right; keep the maximum.
5. Print the maximum.

6 Hand trace

```
lamps: 1 0 1 1 1 0 0 1 1 1 1 0 1      (lamp numbers 1..13)
repair lamp 2  : left 1, right 3  -> 1+1+3 = 5
repair lamp 6  : left 3, right 0  -> 3+1+0 = 4
repair lamp 7  : left 0, right 4  -> 0+1+4 = 5
repair lamp 12 : left 4, right 1  -> 4+1+1 = 6   <- best
```

7 Python solution

solutions/P02_fix_one_street_lamp.py

```python
lamps = input("Enter lamps: ").strip()
n = len(lamps)
best = 0

for i in range(n):
    if lamps[i] == '0':                # try repairing THIS broken lamp
        left = 0
        j = i - 1
        while j >= 0 and lamps[j] == '1':   # count lit lamps to the left
            left += 1
            j -= 1
        right = 0
        j = i + 1
        while j < n and lamps[j] == '1':    # count lit lamps to the right
            right += 1
            j += 1
        best = max(best, left + 1 + right)

print(f"Longest lit stretch: {best}")
```

Even shorter brute force (fine for N ≤ 100)

```python
lamps = input("Enter lamps: ").strip()
best = 0
for i, ch in enumerate(lamps):
    if ch == '0':
        fixed = lamps[:i] + '1' + lamps[i + 1:]
        best = max(best, max(len(run) for run in fixed.split('0')))
print(f"Longest lit stretch: {best}")
```

8 Will Python be fast enough?

$O(N^2)$ worst case, N = 100 → 10,000 steps.

9 Common mistakes & likely twists

- ✗ `int(input())` destroys leading zeros.
- ✗ Forgetting the `+ 1` for the repaired lamp itself.
- ✗ No bounds check → `lamps[-1]` reads the LAST lamp (Python negative index) instead of stopping.

↪ "repair up to K lamps" → sliding window that allows at most K zeros.

↪ "no repair, just the longest stretch" → `max(len(x) for x in s.split('0'))`.

PRACTICE · ITEM 3 · P03

Rotting Fruit Crate

Medium–Hard | T8 Grid | T10 Simulate until stable | T9 BFS

1 What it really asks

Rot spreads up/down/left/right once per minute. Count how many minutes have at least one new rotten fruit.

2 Exact I/O (copy these strings)

INPUT `Enter rows and columns:` → `R C` on one line

INPUT program prints `Enter the crate:` on its own line, then reads R lines (no prompt)

OUTPUT `Minutes: M`

Sample runs highlighted = what the user types

Sample 1

```
Enter rows and columns: 3 3
Enter the crate:
FFF
FFF
RFF
Minutes: 4
```

Sample 2

```
Enter rows and columns: 2 2
Enter the crate:
RF
FF
Minutes: 2
```

Sample 3

```
Enter rows and columns: 2 3
Enter the crate:
FFF
FFF
Minutes: 0
```

3 Spot the pattern

grid + "each minute it spreads to neighbours" + "**at the same time**" = **simultaneous simulation** (the tip literally gives the algorithm) or **multi-source BFS**.

4 Hidden test cases (from the constraints)

- no rotten fruit → 0
- everything already rotten → 0
- single row or single column (bounds!)
- several rotten fruits spreading at once and meeting

5 Approach — before you code

1. Read the grid as a list of lists of characters.
2. Loop: scan every cell; collect each `F` that has an `R` neighbour into a list. **Don't change the grid yet.**
3. If the list is empty → stop. Otherwise turn all of them into `R` and add 1 to minutes.
4. Print minutes.

6 Hand trace

```
Start   Min 1   Min 2   Min 3   Min 4
FFF     FFF     RFF     RRF     RRR
FFF     RFF     RRF     RRR     RRR
RFF     RRF     RRR     RRR     RRR
Minute 5: nothing new -> stop.  Answer 4
```

7 Python solution

solutions/P03_rotting_fruit_crate.py

```python
R, C = map(int, input("Enter rows and columns: ").split())
print("Enter the crate:")
grid = [list(input().strip()) for _ in range(R)]

minutes = 0
while True:
    to_rot = []                              # step 1: MARK, don't change yet
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 'F':
                for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 'R':
                        to_rot.append((r, c))
                        break
    if not to_rot:                           # step 2: nothing marked -> stop
        break
    for r, c in to_rot:                      # apply all changes at once
        grid[r][c] = 'R'
    minutes += 1

print(f"Minutes: {minutes}")
```

Faster version: multi-source BFS (use when the grid is big) — solutions/P03b_rotting_fruit_bfs.py

```python
# Same problem, solved with multi-source BFS (faster for big grids)
from collections import deque

R, C = map(int, input("Enter rows and columns: ").split())
print("Enter the crate:")
grid = [list(input().strip()) for _ in range(R)]

dist = [[-1] * C for _ in range(R)]
q = deque()
for r in range(R):
    for c in range(C):
        if grid[r][c] == 'R':
            dist[r][c] = 0          # every rotten fruit is a starting point
            q.append((r, c))

minutes = 0
while q:
    r, c = q.popleft()
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 'F' and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1
            minutes = max(minutes, dist[nr][nc])
            q.append((nr, nc))

print(f"Minutes: {minutes}")
```

8 Will Python be fast enough?

Grid ≤ 10×10. Simulation: ≤ 100 minutes × 100 cells. BFS is O(R·C) — needed only for big grids.

9 Common mistakes & likely twists

- ✗ Changing cells **during** the scan → a fruit that just rotted spreads in the same minute → answer too small.
- ✗ Neighbour `r-1 = -1` → `grid[-1]` is the LAST row in Python. Always check `0 <= nr < R`.
- ✗ Reading `3 3` with two `input()` calls — they are on ONE line.
- ✗ Forgetting `print("Enter the crate:")`.

- ↪ empty cells `.` and "print -1 if some fruit never rots" → BFS, then check for leftover `F`.
- ↪ 8 directions → add the 4 diagonal moves to DIRS.

PRACTICE · ITEM 4 · P04

Calculator Without Zeros

Easy | T2 Digits / string

1 What it really asks

Print N with every `0` digit removed.

2 Exact I/O (copy these strings)

INPUT `Enter number:` → N

OUTPUT `Display: D`

Sample runs highlighted = what the user types

Sample 1
```
Enter number: 40507
Display: 457
```

Sample 2
```
Enter number: 1000000000
Display: 1
```

Sample 3
```
Enter number: 98765
Display: 98765
```

3 Spot the pattern

Anything about **digits** → treat the number as a **string**.

4 Hidden test cases (from the constraints)

- no zeros → unchanged
- `1000000000` → `1`
- zeros in the middle, at the end, in long runs

5 Approach — before you code

1. Read N as a string.
2. `s.replace('0', '')`.
3. Print.

6 Hand trace

```
'40507'.replace('0', '') -> '457'
```

7 Python solution

solutions/P04_calculator_without_zeros.py

```python
n = input("Enter number: ").strip()     # keep it as a STRING
print(f"Display: {n.replace('0', '')}")
```

8 Will Python be fast enough?

At most 10 characters.

9 Common mistakes & likely twists

✗ Doing it with `% 10` math and rebuilding the number backwards — slower to write, easy to reverse by accident.

↪ remove a given digit d → `replace(str(d), '')`.

↪ "largest number after removing one digit" → try every removal: `max(int(s[:i] + s[i+1:]) for i in range(len(s)))`.

PRACTICE · ITEM 5 · P05

Longest Word Length

Easy | T3 Strings

1 What it really asks

Length of the longest word in a sentence.

2 Exact I/O (copy these strings)

INPUT `Enter sentence:` → a line with spaces

OUTPUT `Longest word length: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter sentence: i love programming in c
Longest word length: 11
```

Sample 2
```
Enter sentence: extraordinary day
Longest word length: 13
```

Sample 3
```
Enter sentence: hello
Longest word length: 5
```

3 Spot the pattern

"words separated by spaces" → `split()`.

4 Hidden test cases (from the constraints)

- single word, no spaces
- longest word is the first or the last
- ties between words

5 Approach — before you code

1. `words = sentence.split()`
2. `max(len(w) for w in words)`
3. Print.

6 Hand trace

```
['i', 'love', 'programming', 'in', 'c'] -> lengths 1, 4, 11, 2, 1 -> 11
```

7 Python solution

solutions/P05_longest_word_length.py

```python
sentence = input("Enter sentence: ")
longest = max(len(word) for word in sentence.split())
print(f"Longest word length: {longest}")
```

8 Will Python be fast enough?

200 characters. Instant.

9 Common mistakes & likely twists

✗ C-style manual counting that forgets the LAST word (no space after it). `split()` avoids this.

✗ `split(' ')` returns empty strings if there are double spaces.

↪ print the longest word itself → `max(words, key=len)` (returns the first one on ties).

↪ count words → `len(words)`; shortest → `min(...)`.

PRACTICE · ITEM 6 · P06

Round to the Nearest Ten

Easy | T1 Formula | Python trap

1 What it really asks

Round N to the nearest multiple of 10; last digit 5 rounds up.

2 Exact I/O (copy these strings)

INPUT `Enter number:` → N

OUTPUT `Rounded: R`

Sample runs highlighted = what the user types

Sample 1
```
Enter number: 247
Rounded: 250
```

Sample 2
```
Enter number: 5
Rounded: 10
```

Sample 3
```
Enter number: 1234
Rounded: 1230
```

3 Spot the pattern

Rounding with a **fixed rule** → integer math. **Never** Python's `round()` here.

4 Hidden test cases (from the constraints)

- N < 5 → 0
- N = 5 → 10
- already a multiple of 10 → unchanged
- 999 → 1000 (extra digit)

5 Approach — before you code

1. Add 5, then drop the last digit: `(N + 5) // 10 * 10`.
2. Or follow the statement: if `N % 10 >= 5` → `N - N % 10 + 10`, else `N - N % 10`.

6 Hand trace

```
247 + 5 = 252 -> 252 // 10 = 25 -> 25 * 10 = 250
  5 + 5 = 10  -> 1 -> 10
1234 + 5 = 1239 -> 123 -> 1230
```

7 Python solution

solutions/P06_round_to_nearest_ten.py

```python
N = int(input("Enter number: "))

# Integer trick: add 5, then chop the last digit
rounded = (N + 5) // 10 * 10

# Same thing, written as the problem describes it:
# if N % 10 >= 5: rounded = N - N % 10 + 10
# else:           rounded = N - N % 10
print(f"Rounded: {rounded}")
```

8 Will Python be fast enough?

O(1).

9 Common mistakes & likely twists

✗ `round(25, -1)` gives **20** and `round(5, -1)` gives **0** in Python (round-half-to-even). Hidden test N = 5 fails.

↪ nearest hundred → `(N + 50) // 100 * 100`.

↪ always round down → `N // 10 * 10`; always up → `-(-N // 10) * 10`.

PRACTICE · ITEM 7 · P07

Warmer Than Yesterday

Easy | T5 Neighbours

1 What it really asks

Count days whose temperature is strictly higher than the previous day.

2 Exact I/O (copy these strings)

INPUT `Enter N:` → N

INPUT `Enter the temperatures:` → N numbers on one line

OUTPUT `Warmer days: W`

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 6
Enter the temperatures: 28 30 29 29 31 33
Warmer days: 3
```

Sample 2

```
Enter N: 1
Enter the temperatures: 25
Warmer days: 0
```

Sample 3

```
Enter N: 5
Enter the temperatures: -2 0 4 5 9
Warmer days: 4
```

3 Spot the pattern

"higher than the day **before**" = compare each element with its left neighbour.

4 Hidden test cases (from the constraints)

- N = 1 → 0
- equal days do NOT count
- negative temperatures

5 Approach — before you code

1. Loop `i` from 1 to N-1.
2. If `t[i] > t[i-1]` add 1.
3. Print the count.

6 Hand trace

```
28 30 29 29 31 33
   +     -  =  +  +   -> 3
```

7 Python solution

solutions/P07_warmer_than_yesterday.py

```python
N = int(input("Enter N: "))
temps = list(map(int, input("Enter the temperatures: ").split()))

warmer = 0
for i in range(1, N):
    if temps[i] > temps[i - 1]:      # strictly higher than the day before
        warmer += 1

# Pythonic: warmer = sum(1 for a, b in zip(temps, temps[1:]) if b > a)
print(f"Warmer days: {warmer}")
```

8 Will Python be fast enough?

N ≤ 50.

9 Common mistakes & likely twists

- ✗ Starting at `i = 0` → compares day 1 with `t[-1]` (the last day!).
- ✗ Using `>=` — equal is not warmer.
- ↪ longest warming streak (drill D03).
- ↪ biggest one-day jump → `max(b - a for a, b in zip(t, t[1:]))`.

PRACTICE · ITEM 8 · P08

Library Late Fines

Easy | T7 Clamping

1 What it really asks

Each book: 5 pesos per day beyond day 7, capped at 50 per book. Print the total.

2 Exact I/O (copy these strings)

INPUT `Enter number of books:` → N

INPUT `Enter days kept:` → N numbers

OUTPUT `Total fine: F`

Sample runs highlighted = what the user types

Sample 1
```
Enter number of books: 5
Enter days kept: 3 9 7 20 12
Total fine: 85
```

Sample 2
```
Enter number of books: 3
Enter days kept: 2 7 5
Total fine: 0
```

Sample 3
```
Enter number of books: 1
Enter days kept: 17
Total fine: 50
```

3 Spot the pattern

"no fine if ≤ 7" = floor at 0 → `max(0, …)`. "never more than 50" = cap → `min(50, …)`.

4 Hidden test cases (from the constraints)

- exactly 7 days → 0
- 17 or more days → exactly 50
- all books on time → 0
- N = 1

5 Approach — before you code

1. For each D: `fine = min(50, max(0, D - 7) * 5)`.
2. Add to total, print.

6 Hand trace

```
3 -> 0 | 9 -> 2*5 = 10 | 7 -> 0 | 20 -> 65 capped 50 | 12 -> 25    total 85
```

7 Python solution

solutions/P08_library_late_fines.py

```python
N = int(input("Enter number of books: "))
days = list(map(int, input("Enter days kept: ").split()))

total = 0
for d in days:
    fine = max(0, d - 7) * 5      # 5 pesos per day beyond the 7th (never negative)
    fine = min(fine, 50)          # cap at 50 pesos per book
    total += fine

print(f"Total fine: {total}")
```

8 Will Python be fast enough?

N ≤ 100.

9 Common mistakes & likely twists

✗ `(D - 7) * 5` for D < 7 is **negative** and lowers the total.

✗ Capping the TOTAL instead of each book.

↪ cap on the total → `min(cap, total)` at the end.

↪ second tier (e.g. 10 pesos/day after day 14) → like the water bill.

PRACTICE · ITEM 9 · P09

Plus-Minus Range Sum

Easy T1 Loop

1 What it really asks

A − (A+1) + (A+2) − … up to B.

2 Exact I/O (copy these strings)

INPUT `Enter start:` → A

INPUT `Enter end:` → B

OUTPUT `Alternating sum: S`

Sample runs highlighted = what the user types

Sample 1
```
Enter start: 3
Enter end: 7
Alternating sum: 5
```

Sample 2
```
Enter start: 8
Enter end: 8
Alternating sum: 8
```

Sample 3
```
Enter start: 2
Enter end: 9
Alternating sum: -4
```

3 Spot the pattern

Alternating signs → keep a `sign` variable and flip it each step.

4 Hidden test cases (from the constraints)

- A = B → just A
- result negative
- B = 1,000,000 (loop is still OK in Python: ~0.1 s)

5 Approach — before you code

1. `sign = 1`; for x in A..B: `total += sign * x`; `sign = -sign`.

6 Hand trace

```
2 - 3 + 4 - 5 + 6 - 7 + 8 - 9 = -4
```

7 Python solution

solutions/P09_plus_minus_range_sum.py

```python
A = int(input("Enter start: "))
B = int(input("Enter end: "))

total = 0
sign = 1                    # +1 for the 1st, 3rd, 5th... number
for x in range(A, B + 1):
    total += sign * x
    sign = -sign            # flip every step

print(f"Alternating sum: {total}")
```

8 Will Python be fast enough?

10^6 iterations ≈ 0.1 s. Fine.

9 Common mistakes & likely twists

✗ Deciding the sign with `x % 2` — that only works when A is odd. The sign depends on the **position**, not the number. Sample 3 (A = 2) catches this.

↪ Alternating Factorial Sum (S01) — same pattern with factorials.

PRACTICE · ITEM 10 · P10

Tiered Water Bill

Easy · T7 Tiers

1 What it really asks

First 10 m^3 cost 15 each, the rest 25 each.

2 Exact I/O (copy these strings)

INPUT `Enter cubic meters used:` → U

OUTPUT `Bill: B`

Sample runs highlighted = what the user types

Sample 1
```
Enter cubic meters used: 14
Bill: 250
```

Sample 2
```
Enter cubic meters used: 0
Bill: 0
```

Sample 3
```
Enter cubic meters used: 10
Bill: 150
```

3 Spot the pattern

Tiered pricing → split U into "inside tier" `min(U, 10)` and "above tier" `max(U - 10, 0)`.

4 Hidden test cases (from the constraints)

- U = 0 → 0
- U ≤ 10 (no 25-peso part)
- U = 10 → 150, U = 11 → 175

5 Approach — before you code

1. `bill = min(U, 10) * 15 + max(U - 10, 0) * 25`

6 Hand trace

```
U = 14 -> 10*15 + 4*25 = 150 + 100 = 250
```

7 Python solution

solutions/P10_tiered_water_bill.py

```python
U = int(input("Enter cubic meters used: "))

first = min(U, 10)            # up to 10 m3 at 15 pesos
extra = max(U - 10, 0)        # anything above 10 m3 at 25 pesos
bill = first * 15 + extra * 25

print(f"Bill: {bill}")
```

8 Will Python be fast enough?

O(1).

9 Common mistakes & likely twists

✗ `(U - 10) * 25` without `max(…, 0)` gives a negative extra when U < 10.

↪ three tiers → handle each band: `min(max(U - lo, 0), hi - lo) * rate`.

LAST YEAR · PRELIMS #1 · L01

Alternating Character Pattern

Easy–Medium | T3 Strings | T5 Neighbours

1 What it really asks

Ignoring non-letters, do the letters alternate vowel / consonant (case-insensitive)?

2 Exact I/O (copy these strings)

INPUT `Enter string:` → may contain spaces and symbols

OUTPUT `Result: Alternating` or `Result: Not Alternating`

Sample runs highlighted = what the user types

Sample 1
```
Enter string: bAnAnA
Result: Alternating
```

Sample 2
```
Enter string: apple
Result: Not Alternating
```

Sample 3
```
Enter string: h3ll0 W0rld!
Result: Not Alternating
```

3 Spot the pattern

Filter the letters first, then do a **neighbour comparison** on "is it a vowel?".

4 Hidden test cases (from the constraints)

- no letters at all → Alternating
- exactly one letter → Alternating
- uppercase vowels (`A`)
- symbols/digits/spaces between letters are skipped

5 Approach — before you code

1. `letters = [c.lower() for c in s if c.isalpha()]`
2. For each neighbouring pair: if both vowels or both consonants → Not Alternating.
3. Otherwise Alternating.

6 Hand trace

```
'h3ll0 W0rld!' -> letters h l l w r l d
h(C) l(C) -> same type -> Not Alternating
```

7 Python solution

solutions/L01_alternating_character_pattern.py

```python
s = input("Enter string: ")
VOWELS = "aeiou"

letters = [ch.lower() for ch in s if ch.isalpha()]   # ignore non-letters

ok = True
for i in range(1, len(letters)):
    prev_is_vowel = letters[i - 1] in VOWELS
    curr_is_vowel = letters[i] in VOWELS
    if prev_is_vowel == curr_is_vowel:     # two vowels or two consonants in a row
        ok = False
        break

print("Result: Alternating" if ok else "Result: Not Alternating")
```

8 Will Python be fast enough?

1000 characters.

9 Common mistakes & likely twists

- ✗ Forgetting `.lower()` → `'A' in 'aeiou'` is False.
- ✗ Splitting into words — the pattern continues across spaces.
- ↪ longest alternating run → streak counter (T6).

LAST YEAR · PRELIMS #2 · L02

Peak-Valley Difference

Easy–Medium | T5 Neighbours

1 What it really asks

Sum of peaks − sum of valleys (strictly bigger/smaller than both neighbours; ends never count).

2 Exact I/O (copy these strings)

INPUT `Enter array size:` → N

INPUT `Enter array elements:` → N numbers

OUTPUT `Result: X`

Sample runs highlighted = what the user types

Sample 1

```
Enter array size: 7
Enter array elements: 10 50 20 80 30 90 40
Result: 170
```

Sample 2

```
Enter array size: 5
Enter array elements: 1 2 3 4 5
Result: 0
```

3 Spot the pattern

"both immediate neighbours" → loop `i` from 1 to N-2.

4 Hidden test cases (from the constraints)

- strictly increasing / decreasing → 0
- plateaus (equal neighbours) are neither
- N = 3

5 Approach — before you code

1. For i in 1..N-2: if bigger than both → add to peaks; elif smaller than both → add to valleys.
2. Print `peaks - valleys`.

6 Hand trace

```
10 [50] (20) [80] (30) [90] 40
peaks 50+80+90 = 220, valleys 20+30 = 50 -> 170
```

7 Python solution

solutions/L02_peak_valley_difference.py

```python
N = int(input("Enter array size: "))
arr = list(map(int, input("Enter array elements: ").split()))

peaks = valleys = 0
for i in range(1, N - 1):                # skip first and last element
    if arr[i] > arr[i - 1] and arr[i] > arr[i + 1]:
        peaks += arr[i]
    elif arr[i] < arr[i - 1] and arr[i] < arr[i + 1]:
        valleys += arr[i]

print(f"Result: {peaks - valleys}")
```

8 Will Python be fast enough?

N ≤ 100.

9 Common mistakes & likely twists

✗ `>=` instead of `>` (strictly).

✗ Including index 0 or N-1.

↪ count peaks; print positions of peaks.

LAST YEAR · PRELIMS #3 · L03

Palindromic Prime Counter

Medium | T11 Sieve | T2 Digits

1 What it really asks

How many numbers in [L, R] are both prime and palindromes?

2 Exact I/O (copy these strings)

INPUT `Enter L and R:` → two numbers on ONE line

OUTPUT `Count: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter L and R: 1 10
Count: 4
```

Sample 2
```
Enter L and R: 11 20
Count: 1
```

Sample 3
```
Enter L and R: 100 200
Count: 5
```

3 Spot the pattern

"prime" + range up to 100,000 → **Sieve of Eratosthenes**. Palindrome → `str(x) == str(x)[::-1]`.

4 Hidden test cases (from the constraints)

- L = 1 (1 is NOT prime)
- L = R
- a range with no answers → 0
- R = 100000

5 Approach — before you code

1. Build `is_prime` up to R with a sieve.
2. Loop x from L to R; count if prime and palindrome.

6 Hand trace

```
1..10   -> 2, 3, 5, 7                 = 4
100..200 -> 101, 131, 151, 181, 191   = 5
```

7 Python solution

solutions/L03_palindromic_prime_counter.py

```python
L, R = map(int, input("Enter L and R: ").split())

# Sieve of Eratosthenes up to R
is_prime = [True] * (R + 1)
is_prime[0] = False
if R >= 1:
    is_prime[1] = False
for i in range(2, int(R ** 0.5) + 1):
    if is_prime[i]:
        for j in range(i * i, R + 1, i):
            is_prime[j] = False

count = 0
for x in range(L, R + 1):
    if is_prime[x] and str(x) == str(x)[::-1]:
        count += 1

print(f"Count: {count}")
```

Alternative: cheap check first (only ~1000 palindromes ≤ 100000)

```python
def is_prime(x):
    if x < 2:
        return False
    for d in range(2, int(x ** 0.5) + 1):
        if x % d == 0:
            return False
    return True

L, R = map(int, input("Enter L and R: ").split())
count = sum(1 for x in range(L, R + 1) if str(x) == str(x)[::-1] and is_prime(x))
print(f"Count: {count}")
```

8 Will Python be fast enough?

Sieve to 10^5: ~0.05 s. Trial-dividing EVERY number would be ~10^7 steps — avoid, or check palindrome first.

9 Common mistakes & likely twists

- ✗ Treating 1 as prime.
- ✗ `range(L, R)` misses R.
- ✗ Reading L and R with two `input()` calls.

↪ count primes whose digit sum is prime; twin primes (p and p+2).

LAST YEAR · PRELIMS #4 · L04

Digit Product Sum Series

Medium | T2 Digits | itertools

1 What it really asks

Repeatedly replace N by the sum of products of every pair of its distinct digits until one digit is left; print the sum of all terms.

2 Exact I/O (copy these strings)

INPUT `Enter N:`

OUTPUT `Total sum of sequence: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter N: 234
Total sum of sequence: 274
```

Sample 2
```
Enter N: 12
Total sum of sequence: 14
```

Sample 3
```
Enter N: 10
Total sum of sequence: 10
```

3 Spot the pattern

"repeat until single digit" → `while cur >= 10`. "pairs of digits" → `itertools.combinations(digits, 2)`.

4 Hidden test cases (from the constraints)

- N already one digit → total = N
- a 0 digit (10 → products 0 → sequence 10, 0 → 10)
- repeated digits (55 → only one distinct digit → no pairs → 0)
- the PDF lost superscripts: "1 ≤ N ≤ 10" is really 10^9 or similar — Python handles big ints

5 Approach — before you code

1. `total = N`, `cur = N`.
2. While `cur >= 10`: digits = `set(str(cur))`; `cur = sum(a*b for pairs)`; `total += cur`.
3. Print total.

6 Hand trace

```
234 -> 2*3 + 2*4 + 3*4 = 26
26  -> 2*6 = 12
12  -> 1*2 = 2  (single digit, stop)
234 + 26 + 12 + 2 = 274
```

7 Python solution

solutions/L04_digit_product_sum_series.py

```python
from itertools import combinations

N = int(input("Enter N: "))

def next_term(x):
    digits = sorted(set(str(x)))                  # distinct digits
    return sum(int(a) * int(b) for a, b in combinations(digits, 2))

total = N
cur = N
while cur >= 10:                                  # stop at a single digit
    cur = next_term(cur)
    total += cur

print(f"Total sum of sequence: {total}")
```

8 Will Python be fast enough?

Values shrink very fast (at most 870 after the first step).

9 Common mistakes & likely twists

- ✗ Forgetting to add N itself.
- ✗ Using all digits instead of distinct digits.
- ✗ Ambiguity: if two pairs give the same product (1·6 = 2·3), "unique products" might mean count it once. Our code sums every pair; if hidden tests fail, try `sum(set(products))`.

↪ sum of digit squares until 1 (happy numbers).

LAST YEAR · PRELIMS #5 · L05

Cyclic Array Shift

Easy–Medium | T14 Rotation / modulo

1 What it really asks

Rotate the array K places left or right (wrap around).

2 Exact I/O (copy these strings)

INPUT `Enter array size:` → N

INPUT `Enter shift amount K:` → K (can be HUGE)

INPUT `Enter shift direction (L/R):` → L or R

INPUT `Enter array elements:` → N numbers

OUTPUT `Shifted array: …` space-separated

Sample runs highlighted = what the user types

Sample 1

```
Enter array size: 5
Enter shift amount K: 2
Enter shift direction (L/R): L
Enter array elements: 1 2 3 4 5
Shifted array: 3 4 5 1 2
```

Sample 2

```
Enter array size: 5
Enter shift amount K: 2
Enter shift direction (L/R): R
Enter array elements: 1 2 3 4 5
Shifted array: 4 5 1 2 3
```

Sample 3

```
Enter array size: 7
Enter shift amount K: 10
Enter shift direction (L/R): L
Enter array elements: -5 0 12 34 -99 100 7
Shifted array: 34 -99 100 7 -5 0 12
```

3 Spot the pattern

"wrap around" + "K can be larger than N" → `K %= N` then **slice**.

4 Hidden test cases (from the constraints)

- K = 0
- K a multiple of N → unchanged
- K > N
- N = 1
- negative numbers

5 Approach — before you code

1. `K %= N`
2. Left: `A[K:] + A[:K]`. Right: `A[N-K:] + A[:N-K]`.
3. `print("Shifted array:", *result)`

6 Hand trace

```
K = 10, N = 7 -> K % 7 = 3
left 3: [-5, 0, 12 | 34, -99, 100, 7] -> 34 -99 100 7 -5 0 12
```

7 Python solution

solutions/L05_cyclic_array_shift.py

```
N = int(input("Enter array size: "))
K = int(input("Enter shift amount K: "))
D = input("Enter shift direction (L/R): ").strip().upper()
A = list(map(int, input("Enter array elements: ").split()))

K %= N                          # shifting N times = no change
if D == 'L':
    result = A[K:] + A[:K]
else:                           # right shift by K = left shift by N-K
    result = A[N - K:] + A[:N - K]

print("Shifted array:", *result)
```

8 Will Python be fast enough?

O(N). Never shift one step at a time K times — K may be 10^{18}.

9 Common mistakes & likely twists

- ✗ Looping K times.
- ✗ Printing the list with brackets (`print(result)`).
- ✗ Not stripping the direction character.

↪ rotate a string (same slicing); rotate a matrix (drill D10).

LAST YEAR · PRELIMS #6 · L06

Pascal's Triangle

Easy–Medium | T13 DP (row building)

1 What it really asks

Print the first N rows of Pascal's triangle.

2 Exact I/O (copy these strings)

INPUT `Enter the number of rows (N):`

OUTPUT `Pascal's Triangle with N rows:` then N lines

Sample runs highlighted = what the user types

Sample 1

```
Enter the number of rows (N): 1
Pascal's Triangle with 1 rows:
1
```

Sample 2

```
Enter the number of rows (N): 5
Pascal's Triangle with 5 rows:
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

3 Spot the pattern

Each row is built from the previous one → keep `row`, build `next_row`.

4 Hidden test cases (from the constraints)

- N = 1
- N = 30 → biggest value 77,558,760 (fine in Python)

5 Approach — before you code

1. `row = [1]`
2. Repeat N times: print the row; next = `[1] + [row[i] + row[i+1] …] + [1]`.

6 Hand trace

```
[1] -> [1,1] -> [1,2,1] -> [1,3,3,1] -> [1,4,6,4,1]
```

7 Python solution

solutions/L06_pascals_pyramid.py

```python
N = int(input("Enter the number of rows (N): "))
print(f"Pascal's Triangle with {N} rows:")

row = [1]
for _ in range(N):
    print(*row)
    # next row: 1, then sums of neighbours, then 1
    row = [1] + [row[i] + row[i + 1] for i in range(len(row) - 1)] + [1]
```

8 Will Python be fast enough?

30 rows.

9 Common mistakes & likely twists

- ✗ `print(row)` prints `[1, 2, 1]` with brackets → use `print(*row)`.
- ✗ The PDF's samples 2–3 show a double space (`rows (N)`), sample 1 shows a single space. When samples disagree, follow the statement/first sample, and test both if allowed.

↪ print only row N; centred pyramid → `print(' ' * (N - i - 1) + ' '.join(...))`.

LAST YEAR · PRELIMS #7 · L07

Stable Sequence Reduction

Medium | T10 Simulate until stable

1 What it really asks

Repeatedly merge neighbouring pairs whose difference ≤ T into their average (left-to-right passes) until a pass merges nothing.

2 Exact I/O (copy these strings)

INPUT `Enter the number of elements and the threshold:` → `N T` on one line (T is a float!)

INPUT `Enter the sequence:`

OUTPUT `Final stable sequence: …` each with 2 decimals

Sample runs highlighted = what the user types

Sample 1

```
Enter the number of elements and the threshold: 4 2.0
Enter the sequence: 10 11 13 15
Final stable sequence: 10.50 14.00
```

Sample 2

```
Enter the number of elements and the threshold: 5 3.0
Enter the sequence: 10 12 20 22 30
Final stable sequence: 11.00 21.00 30.00
```

Sample 3

```
Enter the number of elements and the threshold: 4 5.0
Enter the sequence: 10 20 30 40
Final stable sequence: 10.00 20.00 30.00 40.00
```

3 Spot the pattern

"repeat until no more merges in a full pass" → `while True` + `merged` flag. Merging skips ahead 2 → use a **while loop with an index**, not `for`.

4 Hidden test cases (from the constraints)

- T = 0 → only equal neighbours merge
- N = 1
- nothing merges → print originals as `.00`
- merges that only become possible in pass 2, 3, …

5 Approach — before you code

1. One pass: `i = 0`; if `i+1` exists and `|s[i]-s[i+1]| <= T` → append average, `i += 2`; else append `s[i]`, `i += 1`.
2. Repeat passes until a pass makes no merge.
3. Print with `f"{x:.2f}"`.

6 Hand trace

```
pass 1: (10,11) -> 10.5 | (13,15) -> 14     => [10.5, 14]
pass 2: |10.5 - 14| = 3.5 > 2 -> no merge -> stop
```

7 Python solution

solutions/L07_stable_sequence_reduction.py

```python
parts = input("Enter the number of elements and the threshold: ").split()
N = int(parts[0])
T = float(parts[1])
seq = [float(x) for x in input("Enter the sequence: ").split()]

while True:
    new_seq = []
    merged = False
    i = 0
    while i < len(seq):
        if i + 1 < len(seq) and abs(seq[i] - seq[i + 1]) <= T:
            new_seq.append((seq[i] + seq[i + 1]) / 2)   # merge the pair
            i += 2                                      # skip both
            merged = True
        else:
            new_seq.append(seq[i])
            i += 1
    seq = new_seq
    if not merged:                                      # full pass, no merge -> stable
        break

print("Final stable sequence:", " ".join(f"{x:.2f}" for x in seq))
```

8 Will Python be fast enough?

Each pass shrinks the list or stops → ≤ N passes × N = 10^4.

9 Common mistakes & likely twists

- ✗ `int("2.0")` crashes — read T with `float`.
- ✗ Comparing a fresh average with the next number in the SAME pass (the statement continues with the rest of the original).
- ✗ Forgetting the 2-decimal format.

↪ merge into the sum instead of the average; merge only equal neighbours (like 2048).

LAST YEAR · PRELIMS #8 · L08

Maximal Square of Ones

Hard T13 2D DP

1 What it really asks

Side length of the biggest all-1 square in a 0/1 matrix (up to 500×500).

2 Exact I/O (copy these strings)

INPUT `Enter rows and cols:` → `N M`

INPUT program prints `Enter the matrix:`, then N lines of space-separated 0/1

OUTPUT `Largest square side: K`

Sample runs highlighted = what the user types

Sample 1

```
Enter rows and cols: 4 5
Enter the matrix:
1 0 1 1 1
1 1 1 1 1
1 1 1 1 1
1 0 1 1 0
Largest square side: 3
```

Sample 2

```
Enter rows and cols: 1 5
Enter the matrix:
0 1 1 0 1
Largest square side: 1
```

3 Spot the pattern

"largest square" + 500×500 → brute force is too slow → classic **DP**: `dp[i][j]` = biggest square ending (bottom-right corner) at (i, j).

4 Hidden test cases (from the constraints)

- all zeros → 0
- a single row → 0 or 1
- all ones → min(N, M)
- 500×500

5 Approach — before you code

1. If `grid[i][j] == 1`: `dp[i][j] = 1` on the first row/column, else `1 + min(top, left, top-left)`.
2. Track the maximum while filling.
3. Print it.

6 Hand trace

```
grid          dp
1 0 1 1 1     1 0 1 1 1
1 1 1 1 1     1 1 1 2 2
1 1 1 1 1     1 2 2 2 3
1 0 1 1 0     1 0 1 2 0
biggest value in dp = 3
```

7 Python solution

solutions/L08_maximal_square_of_ones.py

```python
N, M = map(int, input("Enter rows and cols: ").split())
print("Enter the matrix:")
grid = [list(map(int, input().split())) for _ in range(N)]

# dp[i][j] = side of the biggest all-1 square whose BOTTOM-RIGHT corner is (i, j)
dp = [[0] * M for _ in range(N)]
best = 0
for i in range(N):
    for j in range(M):
        if grid[i][j] == 1:
            if i == 0 or j == 0:
                dp[i][j] = 1
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
            best = max(best, dp[i][j])

print(f"Largest square side: {best}")
```

8 Will Python be fast enough?

O(N·M) = 250,000 cells ≈ 0.15 s. Checking every square directly would be ~10^{10} — too slow.

9 Common mistakes & likely twists

- ✗ Reading rows as strings when they are space-separated numbers.
- ✗ No `i == 0 or j == 0` case → `dp[-1]` silently reads the last row.
- ✗ Returning 1 for an all-zero grid (start `best = 0`).

↪ count ALL squares of ones → `sum` of all dp values.

↪ largest square of any one value.

LAST YEAR · SEMIFINALS #1 · S01

Alternating Factorial Sum

Easy T1 Loop

1 What it really asks

1! − 2! + 3! − 4! + … ± N!

2 Exact I/O (copy these strings)

INPUT `Enter a positive integer N:`

OUTPUT `The alternating factorial sum is: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter a positive integer N: 1
The alternating factorial sum
is: 1
```

Sample 2
```
Enter a positive integer N: 3
The alternating factorial sum
is: 5
```

Sample 3
```
Enter a positive integer N: 4
The alternating factorial sum
is: -19
```

3 Spot the pattern

Running product (`fact *= k`) + alternating sign.

4 Hidden test cases (from the constraints)

- N = 1
- N = 20 → about -2.3×10^{18} (overflows in C, fine in Python)
- negative answers

5 Approach — before you code

1. `fact = 1`; for k in 1..N: `fact *= k`; add if k odd, subtract if k even.

6 Hand trace

```
1 - 2 + 6 - 24 = -19
```

7 Python solution

solutions/S01_alternating_factorial_sum.py

```python
N = int(input("Enter a positive integer N: "))

total = 0
fact = 1
for k in range(1, N + 1):
    fact *= k                       # build k! from (k-1)!  -- no recomputing
    if k % 2 == 1:
        total += fact
    else:
        total -= fact

print(f"The alternating factorial sum is: {total}")
```

8 Will Python be fast enough?

20 steps.

9 Common mistakes & likely twists

- ✗ Recomputing the factorial from scratch each time (works, but slower to write).
- ✗ Wrong sign on even k.
- ↪ sum of factorials of the digits (strong numbers).

LAST YEAR · SEMIFINALS #2 · S02

Digital Root

Easy T2 Digits

1 What it really asks

Keep summing the digits until one digit remains.

2 Exact I/O (copy these strings)

INPUT `Enter a number:`

OUTPUT `Digital Root: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter a number: 65536
Digital Root: 7
```

Sample 2
```
Enter a number: 99
Digital Root: 9
```

Sample 3
```
Enter a number: 123
Digital Root: 6
```

3 Spot the pattern

"repeat until single digit" → `while n >= 10`.

4 Hidden test cases (from the constraints)

- n = 0 → 0
- already one digit
- n = 2,000,000,000 → 2

5 Approach — before you code

1. While n ≥ 10: `n = sum(int(d) for d in str(n))`.
2. Print n.

6 Hand trace

```
65536 -> 6+5+5+3+6 = 25 -> 2+5 = 7
```

7 Python solution

solutions/S02_digital_root.py

```python
n = int(input("Enter a number: "))

while n >= 10:
    n = sum(int(d) for d in str(n))

# O(1) formula: root = 0 if n == 0 else 1 + (n - 1) % 9
print(f"Digital Root: {n}")
```

8 Will Python be fast enough?

A few steps. O(1) shortcut: `0 if n == 0 else 1 + (n - 1) % 9`.

9 Common mistakes & likely twists

✗ Only summing once (25 instead of 7).

↪ count how many steps it takes (additive persistence).

LAST YEAR · SEMIFINALS #3 · S03

Positional Cipher

Medium | T4 Cipher

1 What it really asks

Shift each letter BACK by key + its index (0-based), wrapping a↔z; keep case; leave other characters alone.

2 Exact I/O (copy these strings)

INPUT `Enter the encoded string:`

INPUT `Enter the key:`

OUTPUT `Decoded string: …`

Sample runs highlighted = what the user types

Sample 1

```
Enter the encoded string: kiqrv
Enter the key: 3
Decoded string: hello
```

Sample 2

```
Enter the encoded string: Mzpr
Enter the key: 10
Decoded string: Code
```

Sample 3

```
Enter the encoded string:
sfeuiy
Enter the key: 0
Decoded string: secret
```

3 Spot the pattern

letter shifting + wrap-around → `ord` / `chr` with `% 26`.

4 Hidden test cases (from the constraints)

- uppercase stays uppercase
- non-letters unchanged
- shift > 26 (key 100 + index 999) → `% 26` handles it
- key 0

5 Approach — before you code

1. For each `(i, ch)` : shift = key + i.
2. Lowercase: `chr((ord(ch) - 97 - shift) % 26 + 97)` ; uppercase with 65.
3. Otherwise keep ch.

6 Hand trace

```
k(10)-3=7 h | i(8)-4=4 e | q(16)-5=11 l | r(17)-6=11 l | v(21)-7=14 o  -> hello
```

7 Python solution

solutions/S03_positional_cipher.py

```python
s = input("Enter the encoded string: ")
key = int(input("Enter the key: "))

out = []
for i, ch in enumerate(s):
    shift = key + i
    if ch.islower():
        out.append(chr((ord(ch) - ord('a') - shift) % 26 + ord('a')))
    elif ch.isupper():
        out.append(chr((ord(ch) - ord('A') - shift) % 26 + ord('A')))
    else:
        out.append(ch)               # non-letters unchanged

print(f"Decoded string: {''.join(out)}")
```

8 Will Python be fast enough?

1000 characters.

9 Common mistakes & likely twists

✗ In C you need `+ 26` before `%`; in Python `%` is already non-negative — don't over-complicate.

✗ Losing the case. Ambiguity: does the index count spaces? We use the position in the whole string.

↪ encode instead (shift forward, `+ shift`); fixed-shift Caesar; Vigenère (shift from a key word).

LAST YEAR · SEMIFINALS #4 · S04

Digit Sum Power Sum

Easy–Medium | T2 Digits | T1 Loop

1 What it really asks

For i = 1..N add (digit sum of i) ^ (number of digits of i).

2 Exact I/O (copy these strings)

INPUT `Enter N:`

OUTPUT `Total sum: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter N: 1
Total sum: 1
```

Sample 2
```
Enter N: 9
Total sum: 45
```

Sample 3
```
Enter N: 20
Total sum: 434
```

3 Spot the pattern

Per-number digit work inside a loop; N ≤ 100,000 → direct loop is fine.

4 Hidden test cases (from the constraints)

- N = 1
- N = 100000 → a 13-digit answer (fine in Python)

5 Approach — before you code

1. For each i: `s = str(i)`; add `sum(digits) ** len(s)`.

6 Hand trace

```
1..9 -> 45
10..19 -> 1² + 2² + … + 10² = 385
20 -> 2² = 4
total = 434
```

7 Python solution

solutions/S04_digit_sum_power_sum.py

```python
N = int(input("Enter N: "))

total = 0
for i in range(1, N + 1):
    digits = str(i)
    S = sum(int(d) for d in digits)   # digit sum
    D = len(digits)                   # digit count
    total += S ** D

print(f"Total sum: {total}")
```

8 Will Python be fast enough?

10^5 numbers × ≤ 6 digits ≈ 0.13 s.

9 Common mistakes & likely twists

✗ `^` is XOR in Python — power is `**`.

↪ Armstrong numbers (drill D13).

LAST YEAR · SEMIFINALS #5 · S05

Roman to Integer

Easy–Medium | T3 Strings | dict

1 What it really asks

Convert a Roman numeral to an integer.

2 Exact I/O (copy these strings)

INPUT `Enter Roman numeral:`

OUTPUT `Integer value: X`

Sample runs highlighted = what the user types

Sample 1
```
Enter Roman numeral: III
Integer value: 3
```

Sample 2
```
Enter Roman numeral: LVIII
Integer value: 58
```

Sample 3
```
Enter Roman numeral: MCMXCIV
Integer value: 1994
```

3 Spot the pattern

symbol → value lookup = **dict**. Subtractive rule = "a smaller value **before** a bigger one is subtracted".

4 Hidden test cases (from the constraints)

- single symbol
- every subtractive pair (IV IX XL XC CD CM)
- 3999 = MMMCMXCIX

5 Approach — before you code

1. For each position: if the next symbol is bigger, subtract this one; else add it.

6 Hand trace

```
M +1000 | C<M −100 | M +1000 | X<C −10 | C +100 | I<V −1 | V +5  = 1994
```

7 Python solution

solutions/S05_roman_to_integer.py

```python
s = input("Enter Roman numeral: ").strip().upper()
VAL = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}

total = 0
for i in range(len(s)):
    # smaller symbol BEFORE a bigger one -> subtract (IV, IX, XL, XC, CD, CM)
    if i + 1 < len(s) and VAL[s[i]] < VAL[s[i + 1]]:
        total -= VAL[s[i]]
    else:
        total += VAL[s[i]]

print(f"Integer value: {total}")
```

8 Will Python be fast enough?

15 characters.

9 Common mistakes & likely twists

✗ Hard-coding all 6 pairs with `replace` — works, but easy to miss one. The look-ahead rule covers all.

↪ integer → Roman: greedy over `[(1000,'M'),(900,'CM'),(500,'D'),…]`.

LAST YEAR · SEMIFINALS #6 · S06

Smallest Missing Subarray Sum

Hard | T12 Subarray sums | set

1 What it really asks

Smallest positive integer that is NOT the sum of any contiguous subarray.

2 Exact I/O (copy these strings)

INPUT `Enter array size:`

INPUT `Enter the array elements:`

OUTPUT `Smallest Missing Sum: X`

Sample runs highlighted = what the user types

Sample 1

```
Enter array size: 4
Enter the array elements: 3 1 2 7
Smallest Missing Sum: 5
```

Sample 2

```
Enter array size: 3
Enter the array elements: 10 2 8
Smallest Missing Sum: 1
```

Sample 3

```
Enter array size: 4
Enter the array elements: 1 2 3 4
Smallest Missing Sum: 8
```

3 Spot the pattern

"**contiguous** subarray sums" + N ≤ 2000 → $O(N^2)$ double loop with a running sum, store sums in a **set**.

4 Hidden test cases (from the constraints)

- no 1 in the array → answer 1
- all ones (N = 2000) → 2001
- huge values

5 Approach — before you code

1. For each start i: running sum s over j = i..N-1; add s to a set.
2. Pruning: all numbers are positive, so s only grows — once s passes N(N+1)/2 + 1 (the most the answer can be), break.
3. Answer: start at 1 and go up while it is in the set.

6 Hand trace

```
[3, 1, 2, 7]
sums: 3 4 6 13 | 1 3 10 | 2 9 | 7  -> {1,2,3,4,6,7,9,10,13}
1 2 3 4 present, 5 missing -> 5
```

7 Python solution

solutions/S06_smallest_missing_subarray_sum.py

```python
N = int(input("Enter array size: "))
arr = list(map(int, input("Enter the array elements: ").split()))

# There are N*(N+1)/2 subarrays, so the answer can never be bigger than that + 1.
LIMIT = N * (N + 1) // 2 + 1

sums = set()
for i in range(N):
    s = 0
    for j in range(i, N):
        s += arr[j]
        if s > LIMIT:          # all numbers are positive -> sums only grow -> stop
            break
        sums.add(s)

ans = 1
while ans in sums:
    ans += 1

print(f"Smallest Missing Sum: {ans}")
```

8 Will Python be fast enough?

≤ 2 million additions ≈ 0.35 s with the break.

9 Common mistakes & likely twists

✗ **Sorting** the array — that's the different *subset* problem; it changes which subarrays exist.

✗ Storing sums in a list and using `in` → $O(N^2)$ lookups each → far too slow.

↪ subset (not contiguous) version: sort, keep `reach`; if `a > reach + 1` answer is `reach + 1`.

LAST YEAR · SEMIFINALS #7 · S07

ZigZag Conversion

Medium | T14 Bouncing index

1 What it really asks

Write the string down-and-up across numRows rows, then read row by row.

2 Exact I/O (copy these strings)

INPUT `Enter string:`

INPUT `Enter number of rows:`

OUTPUT `Converted string: …`

Sample runs highlighted = what the user types

Sample 1

```
Enter string: PAYPALISHIRING
Enter number of rows: 3
Converted string: PAHNAPLSIIGYIR
```

Sample 2

```
Enter string: THISISATEST
Enter number of rows: 8
Converted string: THISITSSAET
```

3 Spot the pattern

Simulate the pen: a row index that **bounces** between 0 and numRows−1.

4 Hidden test cases (from the constraints)

- numRows = 1 → unchanged
- numRows ≥ length → unchanged
- numRows = 2

5 Approach — before you code

1. `rows = [''] * numRows`; r = 0, step = 1.
2. For each char: append to rows[r]; flip step at the top/bottom; `r += step`.
3. Join the rows.

6 Hand trace

```
P   A   H   N      row 0: PAHN
A P L S I I G      row 1: APLSIIG
Y   I   R          row 2: YIR
-> PAHNAPLSIIGYIR
```

7 Python solution

solutions/S07_zigzag_conversion.py

```python
s = input("Enter string: ")
num_rows = int(input("Enter number of rows: "))

if num_rows == 1 or num_rows >= len(s):
    result = s                          # no zigzag possible
else:
    rows = [""] * num_rows
    r, step = 0, 1
    for ch in s:
        rows[r] += ch
        if r == 0:
            step = 1                    # bounce off the top -> go down
        elif r == num_rows - 1:
            step = -1                   # bounce off the bottom -> go up
        r += step
    result = "".join(rows)

print(f"Converted string: {result}")
```

8 Will Python be fast enough?

1000 characters.

9 Common mistakes & likely twists

✗ numRows = 1 without the special case → the index runs off the end (IndexError).

↪ print the zigzag picture itself; reverse it (decode).

LAST YEAR · SEMIFINALS #8 · S08

Largest Symmetric Subgrid

Hard | T15 Brute force + pruning | T8 Grid

1 What it really asks

Biggest k×k block that looks the same after a 180° rotation.

2 Exact I/O (copy these strings)

INPUT `Enter size of matrix:` → N

INPUT program prints `Enter matrix elements:`, then N rows

OUTPUT `Largest symmetric subgrid size: k`

Sample runs highlighted = what the user types

Sample 1

```
Enter size of matrix: 4
Enter matrix elements:
1 2 3 4
5 6 7 8
8 7 6 5
4 3 2 1
Largest symmetric subgrid size: 4
```

Sample 2

```
Enter size of matrix: 5
Enter matrix elements:
9 1 2 3 9
8 1 2 1 8
7 4 5 4 7
6 1 2 1 6
5 1 2 3 5
Largest symmetric subgrid size: 3
```

3 Spot the pattern

"largest … such that …" → try sizes from **biggest to smallest**; the first symmetric block found is the answer. 180° rotation = reverse the row order AND reverse each row.

4 Hidden test cases (from the constraints)

- N = 1 → 1
- every value equal → N
- nothing bigger than 1 → 1 (a 1×1 block is always symmetric)

5 Approach — before you code

1. For k = N down to 1, for every top-left (r, c):
2. cheap check: the two opposite corners must be equal;
3. full check: row i of the block == row (k-1-i) of the block, reversed.
4. Print the first k that works.

6 Hand trace

```
5×5: corners 9 vs 5 -> no.  size 4: no block works.
size 3 at (1,1):  1 2 1      rotated 180°:  1 2 1
                  4 5 4                     4 5 4
                  1 2 1                     1 2 1   -> 3
```

7 Python solution

solutions/S08_largest_symmetric_subgrid.py

```python
N = int(input("Enter size of matrix: "))
print("Enter matrix elements:")
A = [list(map(int, input().split())) for _ in range(N)]
REV = [row[::-1] for row in A]              # each row reversed (pre-computed once)

def symmetric(r, c, k):
    # cheap check first: the two opposite corners must match
    if A[r][c] != A[r + k - 1][c + k - 1]:
        return False
    # row i of the block must equal row (k-1-i) of the block, reversed
    for i in range(k // 2 + 1):
        top = A[r + i][c:c + k]
        bottom_rev = REV[r + k - 1 - i][N - c - k:N - c]
        if top != bottom_rev:
            return False
    return True

def solve():
    for k in range(N, 0, -1):               # try the BIGGEST size first
        for r in range(N - k + 1):
            for c in range(N - k + 1):
                if symmetric(r, c, k):
                    return k                # first hit = the answer
    return 1

print(f"Largest symmetric subgrid size: {solve()}")
```

Easiest-to-write check (slower, good for partial points)

```python
def symmetric(r, c, k):
    block = [A[i][c:c + k] for i in range(r, r + k)]
    return block == [row[::-1] for row in block[::-1]]
```

8 Will Python be fast enough?

Worst case we measured: random 0/1 250×250 → 1.5 s. Most blocks fail at the first corner check.

9 Common mistakes & likely twists

✗ Checking only the main diagonal or only one pair of rows.

✗ Searching from small to big (then you can't stop early).

↪ mirror symmetry (left-right) → each row is a palindrome; 90° symmetry → `block == rot90(block)`.

6 Drill Set

14 fresh problems that use the techniques from Part 4.

New problems in the same style as CodeChum. Do them **under a timer** (≈15 min each). Prompts and labels are exact — practise copying them. Answers are in Part 7.

D01 · Vowel and Consonant Counter

Easy

Count the vowels (a, e, i, o, u) and consonants in a line of text. Case does not matter. Ignore digits, spaces and symbols.

Input / Output

- read the text after the prompt `Enter text:`
- print `Vowels: V` and `Consonants: C` on two lines

Constraints

- 1 ≤ length ≤ 1000
- the text may contain no letters at all (both counts 0)
- `y` is a consonant

Sample runs highlighted = what the user types

Sample 1

```
Enter text: Programming 101!
Vowels: 3
Consonants: 8
```

Sample 2

```
Enter text: AEIOU xyz
Vowels: 5
Consonants: 3
```

Sample 3

```
Enter text: 2026!!
Vowels: 0
Consonants: 0
```

D02 · Second Highest Score

Easy

Print the second highest DISTINCT score. If it does not exist, print `None`.

Input / Output

- `Enter N:` then `Enter scores:` (N integers on one line)
- print `Second highest: X` or `Second highest: None`

Constraints

- 1 ≤ N ≤ 1000
- scores may repeat
- scores may be negative
- all scores may be equal

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 5
Enter scores: 90 85 90 70 85
Second highest: 85
```

Sample 2

```
Enter N: 3
Enter scores: 50 50 50
Second highest: None
```

Sample 3

```
Enter N: 4
Enter scores: -5 -2 -9 -2
Second highest: -5
```

D03 · Longest Warming Streak

Easy

Find the longest run of consecutive days where each day is strictly warmer than the day before. A single day counts as a streak of 1.

Input / Output

- `Enter N:` then `Enter the temperatures:`
- print `Longest streak: L`

Constraints

- 1 ≤ N ≤ 10^5
- equal temperatures break a streak
- the longest streak may be at the very end

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 7
Enter the temperatures: 20 21 22 19 20 21 23
Longest streak: 4
```

Sample 2

```
Enter N: 1
Enter the temperatures: 30
Longest streak: 1
```

Sample 3

```
Enter N: 4
Enter the temperatures: 5 5 5 5
Longest streak: 1
```

D04 · Island Counter

Medium

A map has land `#` and water `.`. Land cells that touch up, down, left or right belong to the same island (diagonals do NOT connect). Count the islands.

Input / Output

- `Enter rows and columns:` → `R C`; then print `Enter the map:` and read R lines
- print `Islands: K`

Constraints

- 1 ≤ R, C ≤ 100
- the map may be all water (0)
- diagonal land cells are separate islands

Sample runs highlighted = what the user types

Sample 1

```
Enter rows and columns: 4 5
Enter the map:
##..#
#...#
..#..
.....
Islands: 3
```

Sample 2

```
Enter rows and columns: 2 2
Enter the map:
..
..
Islands: 0
```

Sample 3

```
Enter rows and columns: 3 3
Enter the map:
#.#
.#.
#.#
Islands: 5
```

D05 · Range Sum Queries

Medium

Given N numbers and Q questions "L R" (1-based, inclusive), print the sum of positions L..R for each question.

Input / Output

- `Enter N:` , `Enter numbers:` , `Enter Q:` , then print `Enter queries:` and read Q lines `L R`
- print `Sum: S` for every query

Constraints

- 1 ≤ N, Q ≤ 10^5 → a loop per query is too slow; use prefix sums
- L may equal R

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 5
Enter numbers: 3 1 4 1 5
Enter Q: 3
Enter queries:
1 3
Sum: 8
2 5
Sum: 11
4 4
Sum: 1
```

D06 · Best K-Day Sales

Medium

Find the largest total of sales over any K consecutive days.

Input / Output

- `Enter N and K:` → two numbers on one line; `Enter sales:`
- print `Best total: X`

Constraints

- 1 ≤ K ≤ N ≤ 10^5
- K may equal N
- sales may be negative (the answer can be negative)

Sample runs highlighted = what the user types

Sample 1

```
Enter N and K: 7 3
Enter sales: 2 5 1 8 2 9 1
Best total: 19
```

Sample 2

```
Enter N and K: 4 4
Enter sales: 1 2 3 4
Best total: 10
```

Sample 3

```
Enter N and K: 5 1
Enter sales: -3 -1 -7 -2 -5
Best total: -1
```

D07 · Balanced Brackets

Medium

Check whether every `(`, `[`, `{` is closed by the matching bracket in the right order. Ignore other characters.

Input / Output

- `Enter expression:`
- print `Result: Balanced` or `Result: Not Balanced`

Constraints

- 1 ≤ length ≤ 10^4
- a closing bracket may come first
- brackets may be left open at the end

Sample runs highlighted = what the user types

Sample 1
```
Enter expression: {[()()]}
Result: Balanced
```

Sample 2
```
Enter expression: ([)]
Result: Not Balanced
```

Sample 3
```
Enter expression: ((a+b)
Result: Not Balanced
```

Sample 4
```
Enter expression: x = (y + 3) * [z]
Result: Balanced
```

D08 · Most Frequent Word

Medium

Find the word that appears most often (case-insensitive). On a tie, choose the alphabetically first word.

Input / Output

- `Enter sentence:`
- print `Most frequent: word (count)` with the word in lowercase

Constraints

- words are separated by spaces
- ties are possible
- `Hello` and `hello` are the same word

Sample runs highlighted = what the user types

Sample 1
```
Enter sentence: the cat and the dog and the bird
Most frequent: the (3)
```

Sample 2
```
Enter sentence: b a b a
Most frequent: a (2)
```

Sample 3
```
Enter sentence: Hello hello World
Most frequent: hello (2)
```

D09 · Run-Length Encoding

Easy

Compress the text by writing each run of the same character as the character followed by the run length.

Input / Output

- `Enter text:`
- print `Encoded: …`

Constraints

- 1 ≤ length ≤ 1000, no spaces
- runs can be longer than 9 (`z12`)
- a single character → `x1`

Sample runs highlighted = what the user types

Sample 1
```
Enter text: aaabccdddd
Encoded: a3b1c2d4
```

Sample 2
```
Enter text: x
Encoded: x1
```

Sample 3
```
Enter text: zzzzzzzzzzz
Encoded: z12
```

D10 · Rotate the Matrix

Easy–Medium

Rotate an N×N matrix 90° clockwise.

Input / Output

- `Enter N:` , then print `Enter the matrix:` and read N rows
- print `Rotated:` then the N rows, space-separated

Constraints

- 1 ≤ N ≤ 100
- N may be 1

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 3
Enter the matrix:
1 2 3
4 5 6
7 8 9
Rotated:
7 4 1
8 5 2
9 6 3
```

Sample 2

```
Enter N: 1
Enter the matrix:
5
Rotated:
5
```

D11 · Grid Paths

Medium–Hard

Count the paths from the top-left to the bottom-right cell moving only RIGHT or DOWN. `#` cells are blocked.

Input / Output

- `Enter rows and columns:` , then print `Enter the grid:` and read R lines
- print `Paths: P`

Constraints

- 1 ≤ R, C ≤ 100 (answers get huge — fine in Python)
- the start or the end may be blocked → 0

Sample runs highlighted = what the user types

Sample 1

```
Enter rows and columns: 3 3
Enter the grid:
...
...
...
Paths: 6
```

Sample 2

```
Enter rows and columns: 3 3
Enter the grid:
...
.#.
...
Paths: 2
```

Sample 3

```
Enter rows and columns: 2 2
Enter the grid:
#.
..
Paths: 0
```

D12 · GCD and LCM of a List

Easy

Print the greatest common divisor and least common multiple of N positive integers.

Input / Output

- `Enter N:` , `Enter numbers:`
- print `GCD: g` and `LCM: l` on two lines

Constraints

- 1 ≤ N ≤ 100
- N may be 1

Sample runs highlighted = what the user types

Sample 1

```
Enter N: 3
Enter numbers: 12 18 24
GCD: 6
LCM: 72
```

Sample 2

```
Enter N: 1
Enter numbers: 7
GCD: 7
LCM: 7
```

Sample 3

```
Enter N: 2
Enter numbers: 5 7
GCD: 1
LCM: 35
```

D13 · Armstrong Numbers in a Range

Easy–Medium

An Armstrong number equals the sum of its digits each raised to the number of digits (153 = $1^3 + 5^3 + 3^3$). Print all Armstrong numbers in [L, R].

Input / Output

- `Enter L and R:` on one line
- print `Armstrong numbers: …` space-separated, or `Armstrong numbers: None`

Constraints

- 1 ≤ L ≤ R ≤ 10^5
- every 1-digit number is an Armstrong number
- the range may contain none

Sample runs highlighted = what the user types

Sample 1

```
Enter L and R: 100 500
Armstrong numbers: 153 370 371
407
```

Sample 2

```
Enter L and R: 1 9
Armstrong numbers: 1 2 3 4 5 6 7
8 9
```

Sample 3

```
Enter L and R: 10 99
Armstrong numbers: None
```

D14 · Spiral Order

Medium

Print the elements of an R×C matrix in clockwise spiral order, starting at the top-left.

Input / Output

- `Enter rows and columns:` , then print `Enter the matrix:` and read R rows
- print `Spiral order: …`

Constraints

- 1 ≤ R, C ≤ 100
- a single row or a single column
- non-square matrices

Sample runs highlighted = what the user types

Sample 1

```
Enter rows and columns: 3 3
Enter the matrix:
1 2 3
4 5 6
7 8 9
Spiral order: 1 2 3 6 9 8 7 4 5
```

Sample 2

```
Enter rows and columns: 3 4
Enter the matrix:
1 2 3 4
5 6 7 8
9 10 11 12
Spiral order: 1 2 3 4 8 12 11 10
9 5 6 7
```

Sample 3

```
Enter rows and columns: 3 1
Enter the matrix:
1
2
3
Spiral order: 1 2 3
```

7 Drill Answer Key

Tested solutions.

Try first! Each answer names the technique so you can review it in Part 4.

D01 · Vowel and Consonant Counter — T3 Strings

solutions/drills/D01_vowel_consonant_counter.py

```
s = input("Enter text: ")

vowels = consonants = 0
for ch in s.lower():
    if ch.isalpha():
        if ch in "aeiou":
            vowels += 1
        else:
            consonants += 1

print(f"Vowels: {vowels}")
print(f"Consonants: {consonants}")
```

D02 · Second Highest Score — set + sorted

solutions/drills/D02_second_highest_score.py

```
N = int(input("Enter N: "))
scores = list(map(int, input("Enter scores: ").split()))

distinct = sorted(set(scores), reverse=True)   # remove duplicates, biggest first
if len(distinct) < 2:
    print("Second highest: None")
else:
    print(f"Second highest: {distinct[1]}")
```

D03 · Longest Warming Streak — T6 Streaks

solutions/drills/D03_longest_warming_streak.py

```
N = int(input("Enter N: "))
t = list(map(int, input("Enter the temperatures: ").split()))

best = cur = 1
for i in range(1, N):
    if t[i] > t[i - 1]:
        cur += 1            # streak continues
    else:
        cur = 1             # streak broken -> restart at this day
    best = max(best, cur)

print(f"Longest streak: {best}")
```

D04 · Island Counter — T9 BFS flood fill

solutions/drills/D04_island_counter.py

```python
from collections import deque

R, C = map(int, input("Enter rows and columns: ").split())
print("Enter the map:")
grid = [input().strip() for _ in range(R)]

seen = [[False] * C for _ in range(R)]
islands = 0
for r in range(R):
    for c in range(C):
        if grid[r][c] == '#' and not seen[r][c]:
            islands += 1                        # new island found -> flood it
            seen[r][c] = True
            q = deque([(r, c)])
            while q:
                x, y = q.popleft()
                for dx, dy in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < R and 0 <= ny < C and grid[nx][ny] == '#' and not seen[nx][ny]:
                        seen[nx][ny] = True
                        q.append((nx, ny))

print(f"Islands: {islands}")
```

D05 · Range Sum Queries — T12 Prefix sums

solutions/drills/D05_range_sum_queries.py

```python
N = int(input("Enter N: "))
a = list(map(int, input("Enter numbers: ").split()))
Q = int(input("Enter Q: "))

# prefix[i] = sum of the first i numbers  ->  sum(a[L..R]) = prefix[R] - prefix[L-1]
prefix = [0] * (N + 1)
for i in range(N):
    prefix[i + 1] = prefix[i] + a[i]

print("Enter queries:")
for _ in range(Q):
    L, R = map(int, input().split())
    print(f"Sum: {prefix[R] - prefix[L - 1]}")
```

D06 · Best K-Day Sales — T12 Sliding window

solutions/drills/D06_best_k_day_sales.py

```python
N, K = map(int, input("Enter N and K: ").split())
sales = list(map(int, input("Enter sales: ").split()))

window = sum(sales[:K])          # first window
best = window
for i in range(K, N):
    window += sales[i] - sales[i - K]   # slide: add new day, drop oldest day
    best = max(best, window)

print(f"Best total: {best}")
```

D07 · Balanced Brackets — Stack

solutions/drills/D07_balanced_brackets.py

```
s = input("Enter expression: ")
PAIR = {')': '(', ']': '[', '}': '{'}

stack = []
ok = True
for ch in s:
    if ch in "([{":
        stack.append(ch)
    elif ch in ")]}":
        if not stack or stack[-1] != PAIR[ch]:
            ok = False
            break
        stack.pop()

if stack:                  # something was opened but never closed
    ok = False

print("Result: Balanced" if ok else "Result: Not Balanced")
```

D08 · Most Frequent Word — Counter + sort key

solutions/drills/D08_most_frequent_word.py

```
from collections import Counter

words = input("Enter sentence: ").lower().split()
cnt = Counter(words)

# highest count first; ties -> alphabetically first word
word, freq = min(cnt.items(), key=lambda kv: (-kv[1], kv[0]))

print(f"Most frequent: {word} ({freq})")
```

D09 · Run-Length Encoding — T6 Runs

solutions/drills/D09_run_length_encoding.py

```
s = input("Enter text: ").strip()

out = []
i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]:   # walk to the end of this run
        j += 1
    out.append(f"{s[i]}{j - i}")
    i = j

print(f"Encoded: {''.join(out)}")
```

D10 · Rotate the Matrix — zip / 2D lists

solutions/drills/D10_rotate_matrix.py

```python
N = int(input("Enter N: "))
print("Enter the matrix:")
m = [input().split() for _ in range(N)]

# 90 degrees clockwise = reverse the row order, then turn columns into rows
rotated = [list(row) for row in zip(*m[::-1])]

print("Rotated:")
for row in rotated:
    print(*row)
```

D11 · Grid Paths — T13 Grid DP

solutions/drills/D11_grid_paths.py

```python
R, C = map(int, input("Enter rows and columns: ").split())
print("Enter the grid:")
g = [input().strip() for _ in range(R)]

# ways[i][j] = number of paths from (0,0) to (i,j) moving only right/down
ways = [[0] * C for _ in range(R)]
for i in range(R):
    for j in range(C):
        if g[i][j] == '#':
            ways[i][j] = 0                      # blocked cell
        elif i == 0 and j == 0:
            ways[i][j] = 1                      # start
        else:
            from_top = ways[i - 1][j] if i > 0 else 0
            from_left = ways[i][j - 1] if j > 0 else 0
            ways[i][j] = from_top + from_left

print(f"Paths: {ways[R - 1][C - 1]}")
```

D12 · GCD and LCM of a List — math.gcd

solutions/drills/D12_gcd_lcm_of_list.py

```python
from math import gcd

N = int(input("Enter N: "))
nums = list(map(int, input("Enter numbers: ").split()))

g = nums[0]
l = nums[0]
for x in nums[1:]:
    g = gcd(g, x)
    l = l * x // gcd(l, x)        # lcm(a, b) = a*b / gcd(a, b)

print(f"GCD: {g}")
print(f"LCM: {l}")
```

D13 · Armstrong Numbers in a Range — T2 Digits

solutions/drills/D13_armstrong_numbers.py

```python
L, R = map(int, input("Enter L and R: ").split())

found = []
for x in range(L, R + 1):
    digits = str(x)
    power = len(digits)
    if sum(int(d) ** power for d in digits) == x:
        found.append(x)

if found:
    print("Armstrong numbers:", *found)
else:
    print("Armstrong numbers: None")
```

D14 · Spiral Order — Boundaries / simulation

solutions/drills/D14_spiral_order.py

```python
R, C = map(int, input("Enter rows and columns: ").split())
print("Enter the matrix:")
m = [input().split() for _ in range(R)]

top, bottom, left, right = 0, R - 1, 0, C - 1
out = []
while top <= bottom and left <= right:
    for j in range(left, right + 1):            # go right along the top row
        out.append(m[top][j])
    top += 1
    for i in range(top, bottom + 1):            # go down the right column
        out.append(m[i][right])
    right -= 1
    if top <= bottom:
        for j in range(right, left - 1, -1):    # go left along the bottom row
            out.append(m[bottom][j])
        bottom -= 1
    if left <= right:
        for i in range(bottom, top - 1, -1):    # go up the left column
            out.append(m[i][left])
        left += 1

print("Spiral order:", *out)
```

8 Final Checklist

Print this page and keep it next to your keyboard.

Before submitting EVERY problem

- ☐ Prompt strings copied exactly (spaces, colons, capitals)?
- ☐ Output label copied exactly?
- ☐ Input read in the right **order** and on the right **lines**?
- ☐ Ran all samples with **Run code**?
- ☐ Ran each constraint bullet as a test (0, 1, all-same, max size, ends)?
- ☐ Language is **Python**, not C?
- ☐ No leftover debug `print`?

When you are stuck

- ☐ Re-read the statement — the tip in the problem often IS the algorithm (Street Lamp, Rotting Fruit).
- ☐ Solve the sample by hand on paper, then turn your hand steps into code.
- ☐ Write the brute force first. Submit it. Optimise only if time remains.
- ☐ Check the N limit → Rule 4 table → is your approach fast enough?
- ☐ 15 minutes without progress → move on, come back later.

The 12 lines that solve most Easy problems

```
n = int(input("Enter N: "))
a = list(map(int, input("Enter the numbers: ").split()))
s = input("Enter sentence: ")
words = s.split()
digits = [int(d) for d in str(n)]
best = max(len(w) for w in words)
cnt = sum(1 for x, y in zip(a, a[1:]) if y > x)
fine = min(50, max(0, d - 7) * 5)
r = (n + 5) // 10 * 10
print(f"Result: {best}")
print("Shifted array:", *a)
print(f"{x:.2f}")
```

Remember

Exact output → easy points first → constraints are the hidden tests → brute force beats a blank. Good luck, CodeBearers!